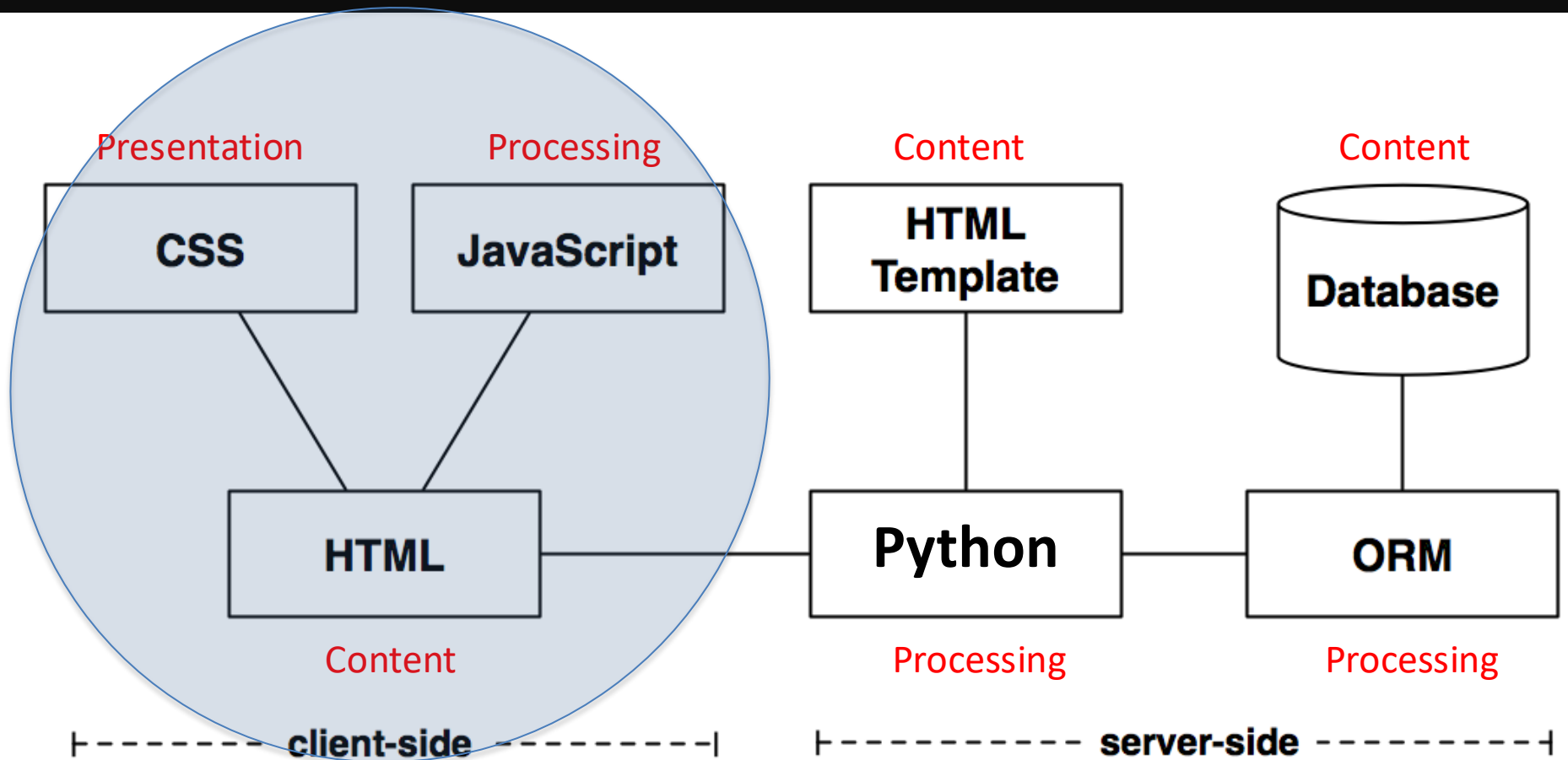


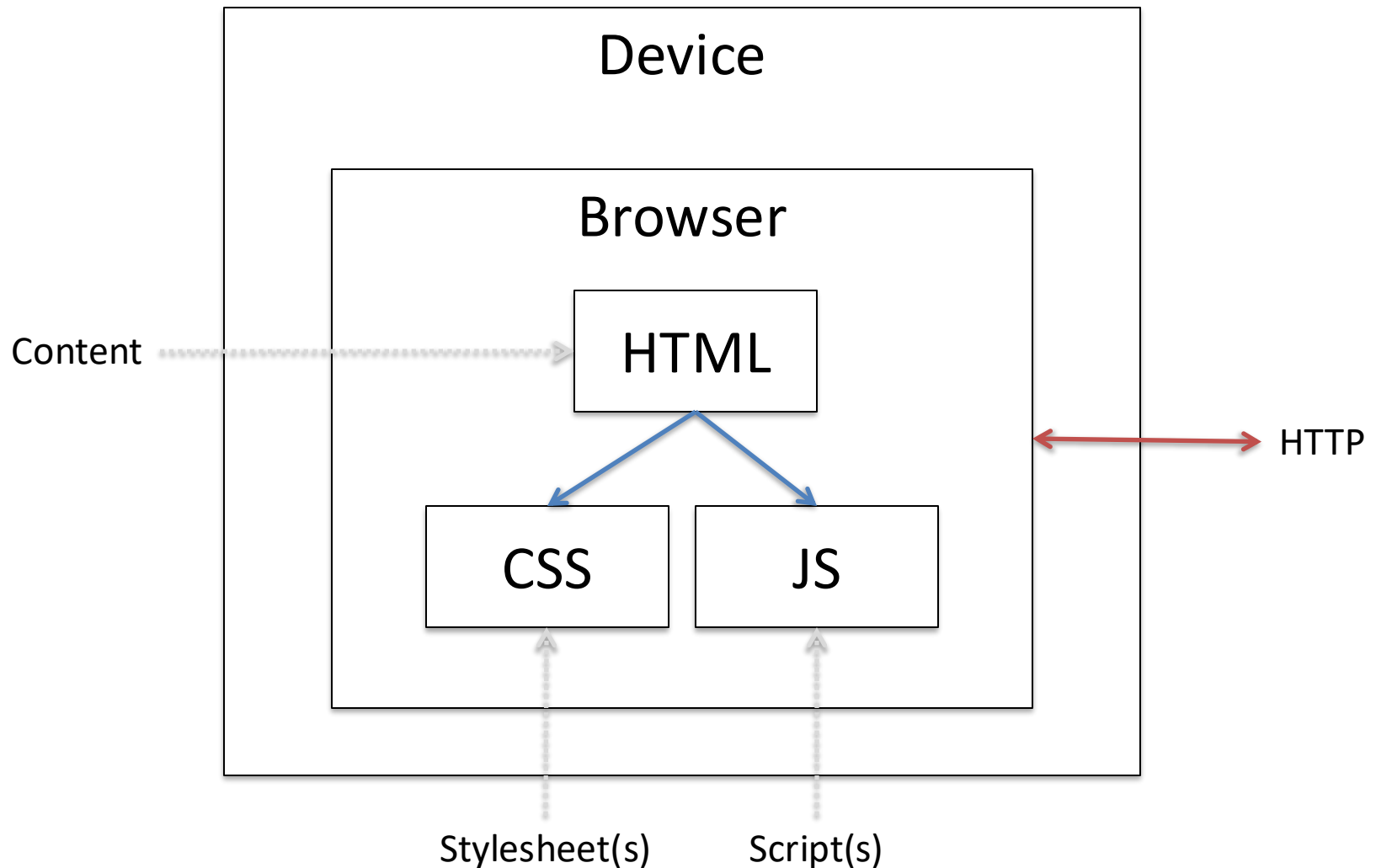
Client-Side Scripting

Internet Technology
ITECH

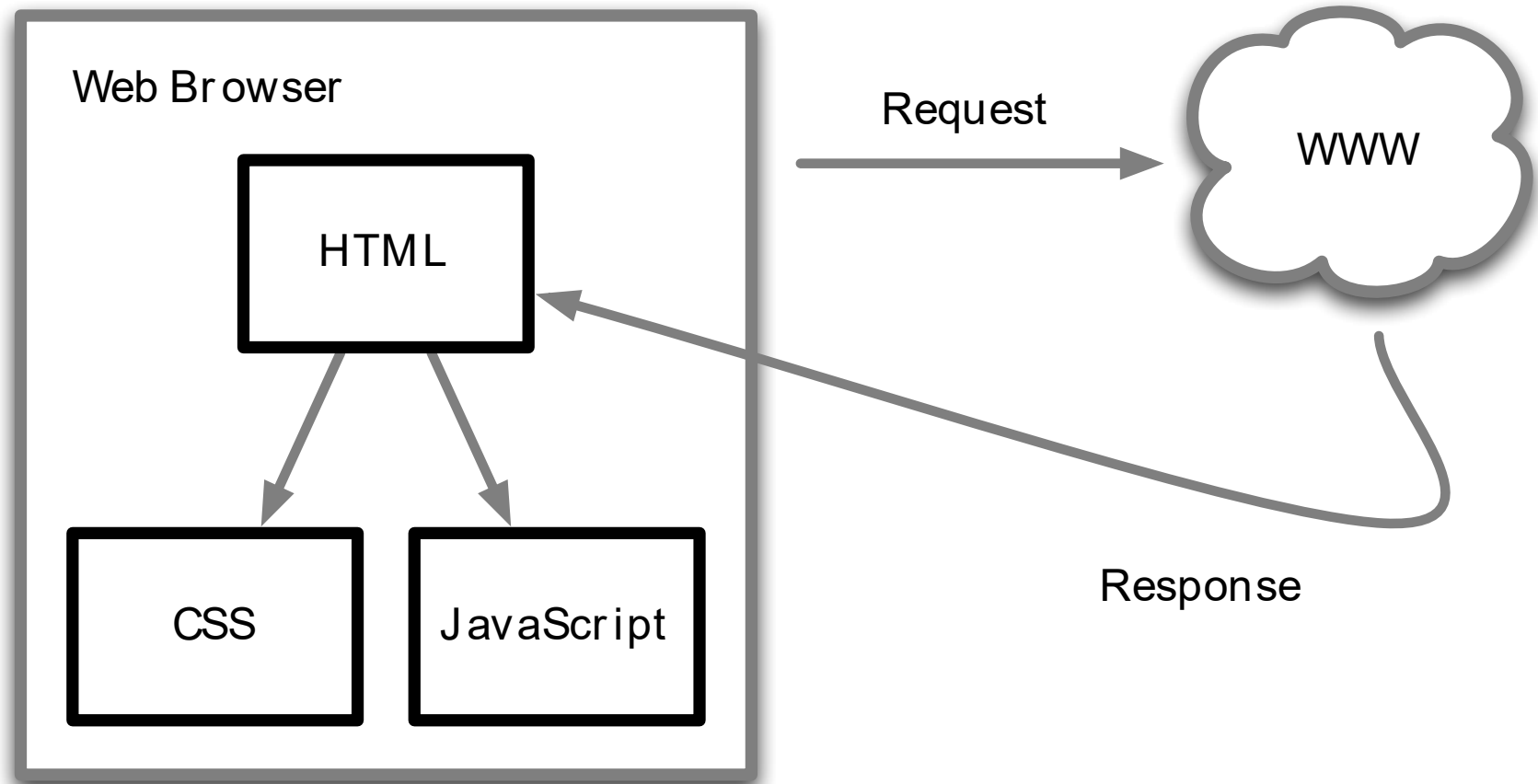
Client-Server model



Client-side Scripting

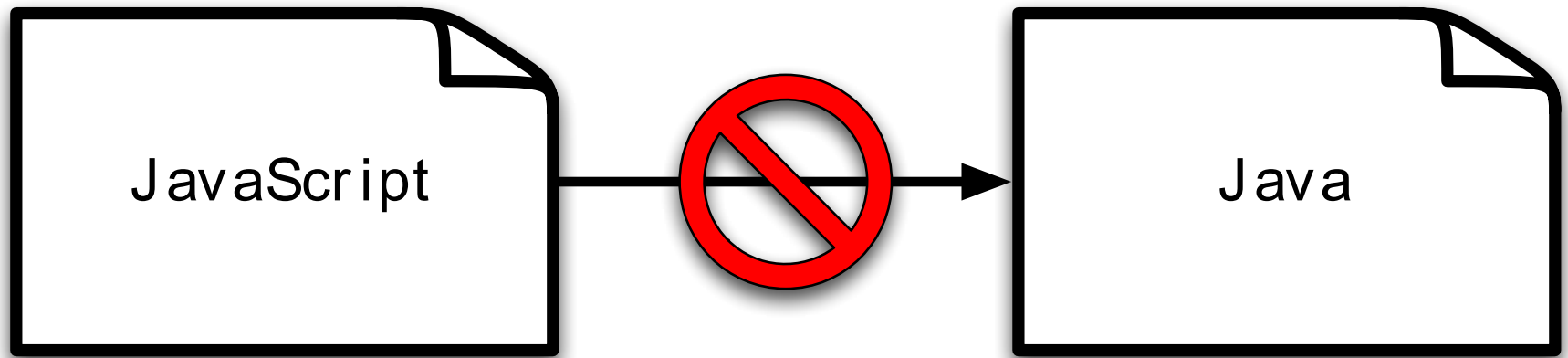


Scripting in Context



JAVASCRIPT

The Name



- It is a completely different language
- Good/bad marketing idea
- Originally called “LiveScript”, but this was not confusing enough

Typecasting

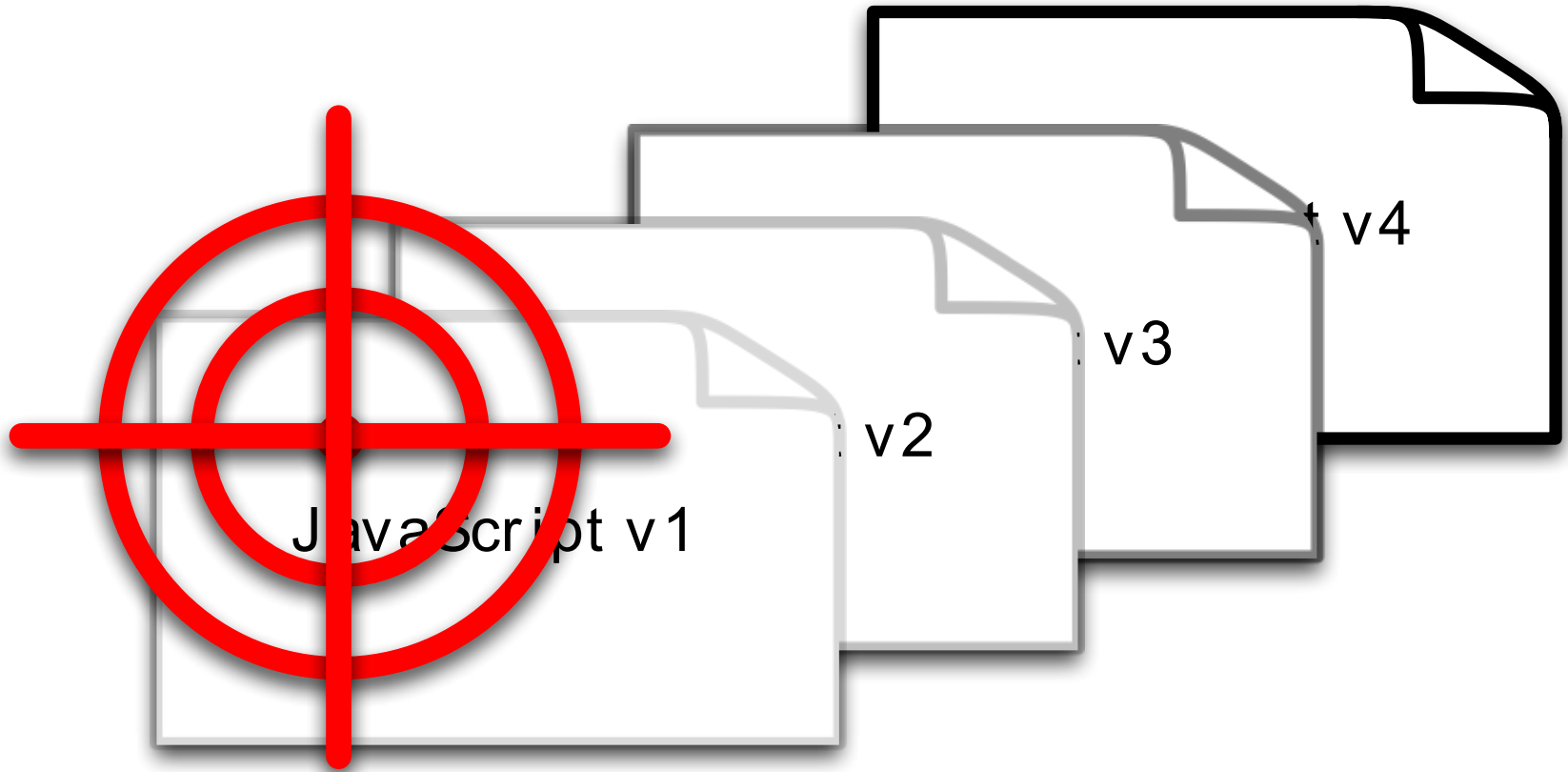


- Designed to run in Netscape Navigator (1995)
- Became standard in all browsers
- Useful for a wide range of programming tasks (*e.g. node.js*)

Procedural and functional

- Javascript looks like a procedural language, but is closer to functional
- Functions are first class
- Supports anonymous functions (*heavily used by jQuery*)

Moving Target



- Opinions formed on earlier versions
- Lacked object-orientation and exception handling
- There is a standard (ECMA) - [ECMA-262, 15th edition, June 2024](#)

Brief history of the standard

- 1st edition: 1997
- 3rd edition: 1999
- 4th edition – abandoned
- 5th edition: 2009
- 6th edition: 2015 *ES6 Harmony*
 - *Supported in more than 98% of browsers*
- 16th edition: 2025



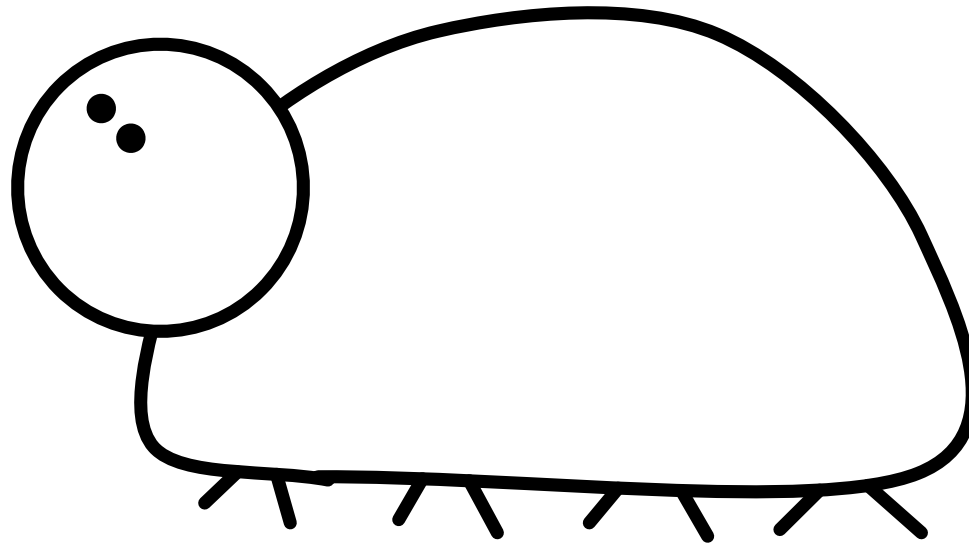
Design Errors

- No language is perfect (that's why there are so many)
- Small things annoy (semi-colon insertion, overloaded operators)
- Problems can be avoided by using a linter JSLint
 - JSLint: <http://www.jshint.com/>
 - ESLint: <https://eslint.org>

Object-Oriented

- Is it Object-Oriented?
 - Yes, has objects, which encapsulate data/methods
- Where are the classes? Where is inheritance?
 - Prototype-based object-oriented language
 - A class in Javascript is a special function
 - The “**class**” keyword is part of ES6
 - Also inheritance - “**extends**” keyword

Lousy Implementations

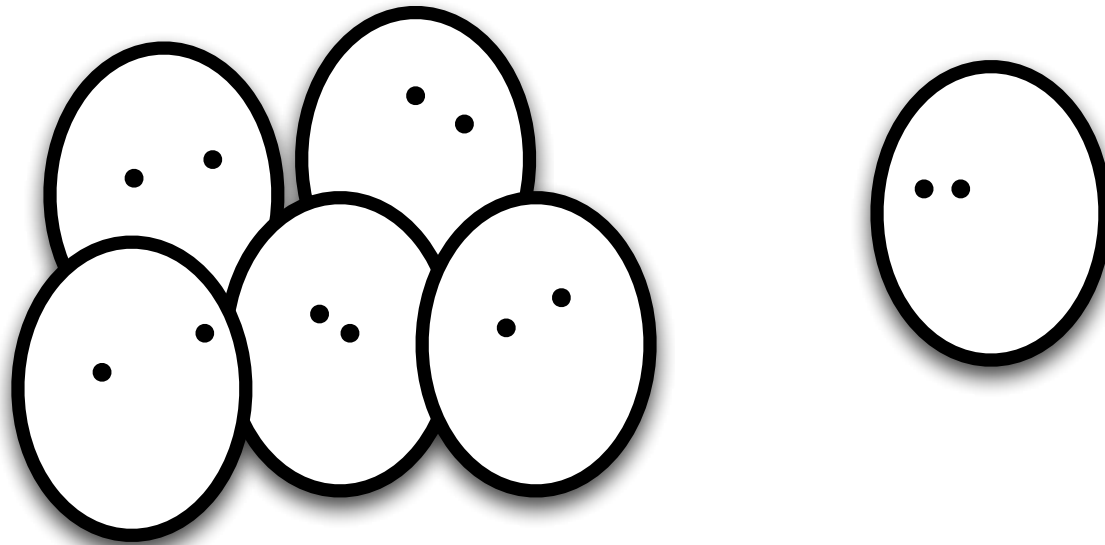


- The JavaScript engines of early browsers were buggy
- The browsers containing JavaScript engines were buggy
- JavaScript performance war has helped

Bunch of Amateurs

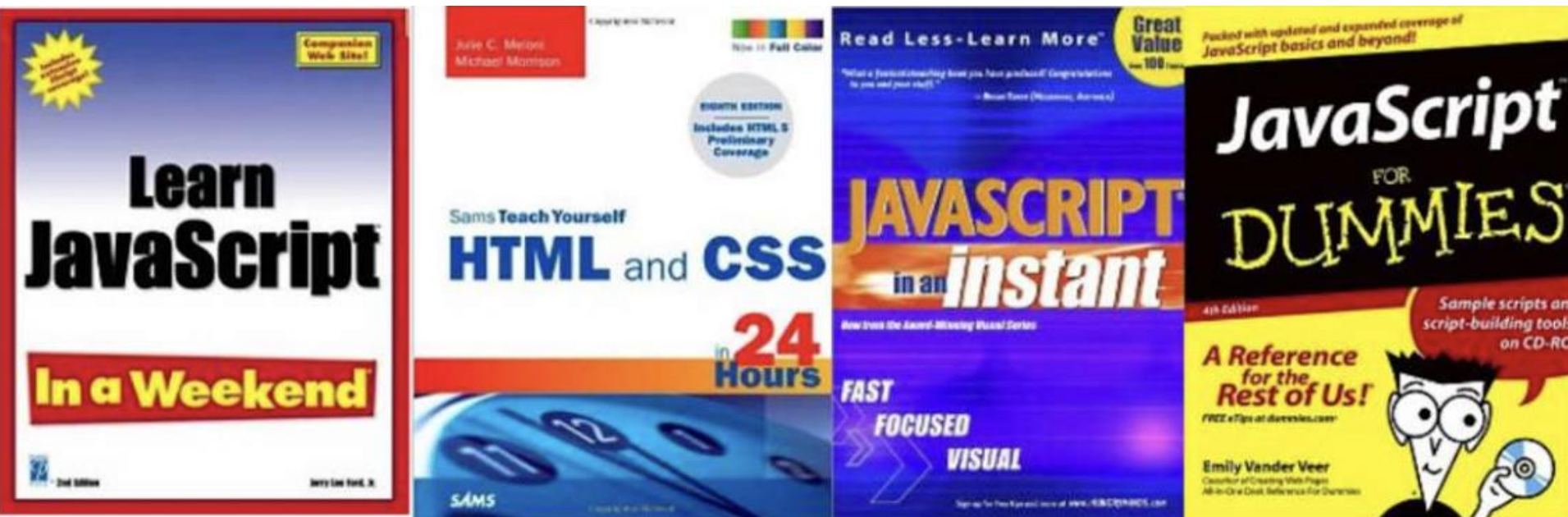
??

!



- Most people writing JavaScript are not programmers
- Lack of training, discipline, common sense
- Expressive language that is severely underutilized

Bad books



THE MISUNDERSTOOD LANGUAGE



"[frustration](#)" by [Sean MacEntee](#) is licensed under [CC BY 2.0](#).

Popularity 2011

Position Feb 2011	Position Feb 2010	Delta in Position	Programming Language	Ratings Feb 2011	Delta Feb 2010	Status
1	1	=	Java	18.482%	+1.13%	A
2	2	=	C	14.986%	-1.62%	A
3	4	↑	C++	8.187%	-1.26%	A
4	7	↑↑↑	Python	7.038%	+2.72%	A
5	3	↓↓	PHP	6.973%	-3.03%	A
6	6	=	C#	6.809%	+1.79%	A
7	5	↓↓	(Visual) Basic	4.924%	-2.13%	A
8	12	↑↑↑↑	Objective-C	2.571%	+0.79%	A
9	10	↑	JavaScript	2.558%	-0.08%	A
10	8	↓↓	Perl	1.907%	-1.69%	A

<http://www.tiobe.com/index.php/content/paperinfo/tpci/index.html>

Popularity 2024

Feb 2024	Feb 2023	Change	Programming Language		Ratings	Change
1	1			Python	15.16%	-0.32%
2	2			C	10.97%	-4.41%
3	3			C++	10.53%	-3.40%
4	4			Java	8.88%	-4.33%
5	5			C#	7.53%	+1.15%
6	7	^		JavaScript	3.17%	+0.64%
7	8	^		SQL	1.82%	-0.30%
8	11	^		Go	1.73%	+0.61%
9	6	v		Visual Basic	1.52%	-2.62%
10	10			PHP	1.51%	+0.21%

<http://www.tiobe.com/tiobe-index>

Core Features

- Syntactically similar to Java/C
(if/else, while, for)
- Familiar primitive datatypes
(numbers, strings, Booleans)
- Object-oriented
(in its own way)

Core Features

- Interpreted Language
(no compiling)
- Dynamic Typing
(var x = 10; var y = "abc";)
- Functions are first class
(can also be anonymous and nested)

Inline JavaScript

- Scripts can be included inline with HTML code
- Good for experimentation
- Violates separation of concerns

```
<html>
  <head>
  </head>
  <body>
    <script type="text/javascript">
      document.writeln("Hello World!");
    </script>
  </body>
</html>
```

- Scripts can also be added to event handlers
- Fragile to maintain

External JavaScript

- Scripts can be kept in external files and linked to from the <head> section
- Easier to manage code (over time)

```
<html>  
  <head>  
    <script type="text/javascript" src="myScript.js"></script>  
  </head>  
  <body>  
  </body>  
</html>
```

DOM Integration

- The intent behind JavaScript was to dynamically script/manipulate documents
- HTML documents are modelled using DOM
- DOM methods and properties can be accessed and altered using JavaScript

Finding Elements in the DOM

- Finding DOM elements to manipulate
- `getElementsByTagName( )`
- `getElementById( )`

```
// Find the number of tables in a document
```

```
var tables = document.getElementsByTagName("table");  
alert("This document contains " + tables.length + " tables");
```

```
// Find a specific Table within a document and count its rows
```

```
var tableOfContents = document.getElementById("TOC");  
var rows = tableOfContents.getElementsByTagName("tr");  
var numrows = rows.length;
```


Modifying Elements in the DOM

- The real impact of JavaScript is changing the content of the DOM

```
// This function traverses the DOM tree and
// converts all Text node data to uppercase
function upcase(n) {
  if (n.nodeType == 3 /*Node.TEXT_NODE*/) {
    n.nodeValue = n.nodeValue.toUpperCase();
  } else {
    // If the node is not Text, loop through its children
    // and recursively call this function on each child.
    var kids = n.childNodes;
    for (var i = 0; i < kids.length; i++) {
      upcase(kids[i]);
    }
  }
}
```

Modifying Elements in the DOM

- `nodeType` returns the type of the node
 - 1 for an element node
 - 2 for an attribute node
 - 3 for a text node
 - 8 for a comment node
 - 9 for a document node
- Include a reference to script containing `uppercase` in html head
- Call this function by putting the following at the bottom of the document body:

```
<script type="text/javascript">  
    uppercase(document.body)  
</script>
```

← TRY THIS

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes and values
- Variables
- Expressions and Operators
- Statements
- Objects and Arrays
- Functions
- Class, Constructors and Prototypes
- Pattern Matching and Regular Expressions

```
if (xo.status == 200) {  
    xa.open();  
    xa.type = 1;  
    xa.write(xo.responseBody);  
    if (xa.size > 1000) {  
        dn = 1;  
        xa.position = 0;  
        xa.function j3() {
```

FOOD FOR THOUGHT...

The Problem

- Despite JavaScript and DOM being functionally useful, coding on the client-side is not particularly easy
- Consider Java and its huge standard library of useful functionality
- DOM scripting entails a lot of repetitive domain-specific boilerplate coding

A Solution

- JavaScript needs its own standard library
- Focus on the domain-specific programming tasks (user interaction, animation, etc)
- More than one solution: *jQuery, AngularJS, React, node.js, etc*

Resources

Overview

<https://jgthms.com/javascript-in-14-minutes/>

W3Schools JavaScript Tutorial

<http://www.w3schools.com/js/default.asp>

JavaScript: The Definitive Guide

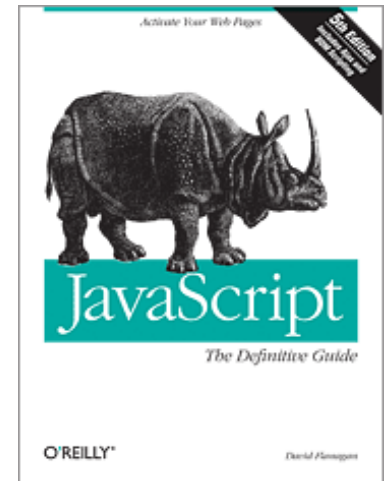
D. Flanagan

<http://oreilly.com/catalog/9780596101992>

JavaScript: The Good Parts

D. Crockford

<http://oreilly.com/catalog/9780596517748/>



Other Resources

Search for **Douglas Crockford & JavaScript** for some video lectures by JavaScript's cranky evangelist:

- The World's Most Misunderstood Language
- The Good Parts
- 4 Part Series for Beginners
- Advanced JavaScript



SYNTAX

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes
- Variables
- Expressions and Operators
- Statements
- Objects and Functions
- Class and Constructors
- Pattern Matching and Regular Expressions

Factorials (N!)

CodeExa
mples:
Factorial

```
<html>
  <head>
    <title>Factorials</title>
  </head>
  <body>
    <h3>Table of Factorials</h3>
    <p id="demo"></p>
    <script type="text/javascript" src="factorial.js"></script>
  </body>
</html>
```

```
var fact = 1;
var text = "";
for (var i = 1; i < 10; i++) {
  fact *= i;
  text += (i + "! = " + fact + "<br />");
}
document.getElementById("demo").innerHTML =
text;
```

factorial.js

Button Event

Code
Examples:
Button

```
<html>
  <head>
    <title>Button Event</title>
  </head>
  <body>
    <h3>Button Event</h3>
    <button onClick="alert('You clicked the button');">
      Click Here
    </button>
  </body>
</html>
```

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes
- Variables
- Expressions and Operators
- Statements
- Objects and Functions
- Class and Constructors
- Pattern Matching and Regular Expressions

Lexical Structure

- JavaScript is a case-sensitive language (keywords, identifiers, variables, functions etc must be consistent)
- Whitespace is ignored (spaces, tabs and newlines) but see below
- Semi-colons are optional – but it is good practice
 - JavaScript interpreters automatically add them – this is a very bad thing so it is better to be explicit!

<code>return</code>		<code>return;</code>
<code>true;</code>		<code>true;</code>

What happens is that **undefined** is returned instead of **true**

Lexical Structure

- **Comments** can be single `//` or multiline `/* */` C-style
- **Literals** are data values that appear directly in the language: `12`, `1.2`, `"hello"`, `true`, `false`
- **Identifiers** are names for variables and functions
 - First character must be letter, underscore or dollar
 - Remaining characters can include above and numbers
- **Reserved word** set cannot be used as identifier
 - Be careful, JavaScript has an unusually large set of reserved words that may become part of the language in the future

https://www.w3schools.com/js/js_reserved.asp

Literal example

```
<script>
document.write('Boolean(12) is ' + Boolean(12));
document.write('<br>');
document.write('Boolean("Hello") is ' + Boolean("Hello"));
document.write('<br>');
document.write('Boolean(3 > 0.02) is ' + Boolean(3 > 0.02));
document.write('<br>');
document.write("Boolean('false') is " + Boolean('false'));
document.write('<br>');
document.write('Boolean(0) is ' + Boolean(0));
</script>
```

```
<script>
document.getElementById("no1").innerHTML = 100.25;
</script>
<script>
document.getElementById("no2").innerHTML = 56e4;
</script>
```

Code
Examples:
Boolean/
Float
Literals

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes
- Variables
- Expressions and Operators
- Statements
- Objects and Functions
- Class and Constructors
- Pattern Matching and Regular Expressions

Datatypes

- Three primitive types:
 - **Number**: no distinction between integers (123) and decimal (3.14) and floating-point (6.02e23) values
 - **String**: sequence of Unicode letters, digits and punctuation characters delimited by single or double quotes (“Hello!”)
 - **Boolean**: true or false
- Two trivial types:
 - **null**: an assignment value that can represent no value – null is (a placeholder for) an object, technically a primitive
 - **undefined**: variable that has been declared but no value has been assigned to it, or an object property that does not exist

Datatypes: Functions

- A function is a piece of executable code that is defined once, but can be called multiple times
- In other languages, functions or methods are often just a useful syntax construct to gather related code
- In JavaScript, functions are 1st class objects in the language, and can be passed as datatypes
- No return type required in function signature

```
function square(x) {  
    return x * x;  
}  
y = square(4);
```

```
var square = function(x) {  
    return x * x };  
y = square(4);
```

Datatypes: Objects

- An **Object** is a collection of named values
- Named values are known as the object's **Properties**
- Objects are created by invoking a **constructor** or using the **object literal** short-hand syntax:

Code
Examples:
Object 1
Object 2

```
function point(xVal, yVal)
{
    this.x = xVal;
    this.y = yVal;
}
```

```
var point1=new point(2.5,
5.4)
```

```
var point = new Object( );
point.x = 2.5;
point.y = 5.4;
```

```
// same as above
var point = {x:2.5, y:5.4};
```

Datatypes: Arrays

- Arrays are also very similar to Objects, acting as a collection of data values
- For objects, each value has a **name** (obj.x), whilst arrays have an **index** (arr[0]) instead
- The elements in an array do not have to have the **same type** (cf. Java arrays), and their size is dynamic
- **Methods:** join, reverse, sort, concat, slice, splice, push, pop

```
var collection = new Array( );  
collection[0] = 120;  
collection[1] = 'hello!';
```

```
// array literal syntax, same as above  
var collection = [120, 'hello!'];
```

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes and values
- **Variables**
- Expressions and Operators
- Statements
- Class, Constructors and Prototypes
- Pattern Matching and Regular Expressions

Variables

- An **identifier** associated with a **value**
- Used to store and manipulate values in a program
- All variables are **untyped** (weak or loose typing)
- Variables are **declared** using the **var** keyword
 - if this is missing, the variable is global – not recommended
- Scope of variables depends on where they are declared
 - global variables can be seen everywhere
 - variables declared in a function are only visible locally
 - omitting **var** in functions will use matching global variables
 - there is no **block** scope like C/Java languages (e.g. in **for** or **if/else** blocks)

```
var i = 10;  
i = "hello!";
```

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes
- Variables
- Expressions and Operators
- Statements
- Objects and Functions
- Class and Constructors
- Pattern Matching and Regular Expressions

Expressions

- An **expression** is a phrase of code that can be evaluated to produce a **value**

```
1.5           // a numeric literal
"hello!"      // a string literal
True          // a Boolean literal
/java/        // a regular expression
               literal
{x:1.2, y:2}   // an object literal
[1, 2, 3, 4, 5] // an array literal
function(x) {return x*x;} // function literal
sum           // the variable sum
```

Operators

- Simple expressions can be combined by using **Operators**
- JavaScript supports a common set of operators compared to other C/Java languages
 - arithmetic (+), equality (==), relational (>), logical (&&), bitwise (<<)
- Care should be taken when using operators
 - '+' can mean addition or concatenation
 - '==' tests for equality, '===' equality and type

```
if (true == 1)    // evaluates as true
if (true === 1)  // evaluates as false
```

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes
- Variables
- Expressions and Operators
- Statements
- Objects and Functions
- Class and Constructors
- Pattern Matching and Regular Expressions

Statements

```
// expression statement  
var x = 1 + 2;
```

```
// if/else if/else  
// statement  
if (condition) {  
    statements  
}  
else if (condition) {  
    statements  
}  
else {  
    statements  
}
```

```
// while statement  
while (condition) {  
    statements  
}
```

```
// for statement  
for (init ; test; inc) {  
    statements  
}
```

```
// function statement  
function name (args) {  
    statements  
}
```

Statements

```
// try catch finally statement
try {
    // normally this code runs from top to bottom
    // sometimes an exception may be thrown
    // either directly with a throw statement,
    // or indirectly by calling another method
}
catch (e) {
    // the statements here are executed if, and only
    // if, the try statement generated an exception
    // these statements handle the exception somehow
}
finally {
    // the statements here are always executed
    // regardless of what happened in the try block
}
```

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes
- Variables
- Expressions and Operators
- Statements
- **Objects and Functions**
- Class and Constructors
- Pattern Matching and Regular Expressions

Objects

- Unordered collections of properties
- Besides the 'dot' operator to access properties, the [...] operator can be used
 - one way to think of this is like Python Dictionaries or the Java Hashtable class
- As JavaScript is dynamic, objects can have properties added at any time, this is a very convenient method

Objects - example

[Example:](#)
[Objects3](#)

```
<body>
  <script type="text/javascript">
    var addr = "";
    var customer=new Object();
    customer.address0="36 King Street";
    customer.address1="42 Queen Road";
    customer.address2="16 Abbey Hill";
    customer.address3="29 Regent Gardens";
    for (i = 0; i < 4; i++) {
      addr += customer["address" + i] + "<br>";
    }
    document.writeln(addr);
  </script>
</body>
```


Functions

- Functions can also be nested
- Functions support optional arguments
 - if invoked with fewer arguments, undefined is used
- Arguments objects can be used with variable length argument lists
 - e.g. Function object has a property **arguments** which can be inspected to find which and how many arguments were given (e.g. if `(arguments.length != 3) { ... }`)
- Functions that are properties of objects are usually referred to as **methods**

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes
- Variables
- Expressions and Operators
- Statements
- Objects and Functions
- Class and Constructors
- Pattern Matching and Regular Expressions

Classes and Constructors

- Creating a class to model Rectangles

```
class Rectangle {  
    // Define the constructor  
    // Note how it calls a method referred to by "this"  
    constructor (idString, xVal, yVal) {  
        this.id = idString;  
        this.resize(xVal,yVal);  
    }  
  
    // What follows is a method  
    resize (xVal, yVal) {  
        this.x = xVal;  
        this.y = yVal;  
    }  
}
```

Classes and Constructors

- Creating a class to model Rectangles (cont)

Code
Examples:
Rectangle

```
// Here is another method
getArea () {
    return this.x * this.y;
}

// Test out the constructor and methods
var rect = new Rectangle ("Test", 4, 5);
document.writeln(rect.id);
document.writeln(rect.getArea());
rect.resize(6, 7);
document.writeln(rect.getArea());
```

The JavaScript Language

- A simple script
- Lexical Structure
- Datatypes
- Variables
- Expressions and Operators
- Statements
- Objects and Functions
- Class and Constructors
- Pattern Matching and Regular Expressions

Regular Expressions

- A regular expression is an object that describes a pattern of characters that can be used to perform **pattern matching** and **search** and **replace** actions on text
- Often RegExps can be thought of as programs within a program
- However, despite their utility, they can be a documentation nightmare

*Some people, when confronted with a problem, think
"I know, I'll use regular expressions."
Now they have two problems.*

J. Zawinski, '97₆₂

Regular Expressions

- In JavaScript, regular expressions are represented by RegExp objects
- Syntax of a regular expression:
 - */pattern/modifiers;*
 - Example 1:

```
var pattern = /Free/i;
```

 - “Free” is a pattern and “i” is a modifier (case insensitive)
 - Example 2:

```
var pattern = /s$/;
```

 - match any string that ends with ‘s’

Regular Expressions

- RegExp object methods:

- search() – returns starting position of the first match, or -1

- Example:

- ```
var str = "Visit W3Schools";
var n = str.search(/w3schools/i);
returns 6
```

- exec() – returns the first match, or null

- Example:

- ```
var str = "Visit W3Schools";  
var match = /w3schools/i.exec(str);  
returns "W3Schools"
```

- test() – returns true if there is a match, false otherwise

- Example:

- ```
var str = "Visit W3Schools";
var match = /w3schools/i.test(str);
returns "true"
```

CodeExa  
mples:  
RegExp

[http://www.w3schools.com/js/js\\_regexp.asp](http://www.w3schools.com/js/js_regexp.asp)



# Takeaways

- JS is a programming language that is one of the core technologies of the World Wide Web, alongside HTML and CSS.
- JS is a high-level, often just-in-time compiled language that conforms to the ECMAScript standard.
- JS has various important features, such as a simple scripting style, lexical structure and datatypes etc.